Engenharia de Software

Detecção de Bad Smells e Refatoração Segura

Análise estática com ESLint + eslint-plugin-sonarjs e refatoração guiada por testes

Lucas Ferreira Ribeiro
Matrícula: 855036

Repositório: github.com/lucasTOSP/bad-smells-js-refactoring
Novembro de 2026

1. Análise de Smells

A leitura manual de `src/ReportGenerator.js` mostrou que toda a lógica está concentrada em um único método, `generateReport`, com 58 linhas. Destaco três smells principais:

1.1 Método Longo (Long Method)

O método faz quatro coisas diferentes: monta o cabeçalho, filtra os itens conforme o perfil do usuário, formata cada linha e monta o rodapé com o total. Para manutenção, qualquer alteração (por exemplo, mudar a regra de filtragem) exige entender o método inteiro, e há sempre o risco de afetar outra responsabilidade. Para os testes, o único ponto de entrada é o método completo: não dá para testar a regra de visibilidade ou o formato de uma linha isoladamente, e cada combinação de tipo × perfil × faixa de valor precisa de um cenário de ponta a ponta.

1.2 Condicionais Repetidas (Repeated Switches)

A comparação `reportType === 'CSV' / 'HTML'` aparece **8 vezes**, espalhada pelo cabeçalho, pelos dois ramos de perfil e pelo rodapé, e está aninhada dentro de `if (user.role === ...)`, que por sua vez está dentro de um `for`. Para adicionar um formato novo (ex.: PDF) seria preciso editar quatro pontos do método, e esquecer um deles gera um relatório inconsistente. Para os testes, o número de caminhos cresce de forma multiplicativa.

1.3 Código Duplicado (Duplicated Code)

A linha CSV `` `${item.id},${item.name},...` `` está repetida nos ramos ADMIN e USER, e `total += item.value` aparece **4 vezes**. Duplicação faz com que uma correção precise ser replicada à mão. O próprio projeto mostrou esse risco: a linha HTML do ramo ADMIN divergiu da linha do ramo USER e passou a gerar `<tr >` (com um espaço sobrando). A suíte de testes acusou esse defeito já na primeira execução (Figura 1).

Também identifiquei, em menor grau, **números mágicos** (`1000` e `500` sem nome que explique a regra) e um **efeito colateral**: o método altera o objeto recebido (`item.priority = true`), o que pode surpreender quem o chama.

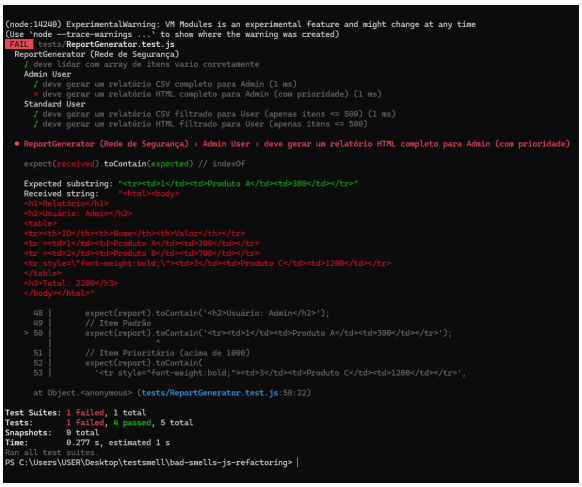


Figura 1 — Primeira execução de npm test no código original: 4 de 5 testes passando. A falha revela o <tr > gerado pelo ramo duplicado.

2. Relatório da Ferramenta (ESLint + SonarJS)

O ESLint foi configurado no `.eslintrc.json` com `eslint:recommended` e `plugin:sonarjs/recommended`. Mantive o limite de complexidade cognitiva do repositório-base (**5**), mais rigoroso que o valor 15 do enunciado. Mesmo com 15, o método original seria reprovado. Resultado de `npx eslint src/` antes da refatoração:

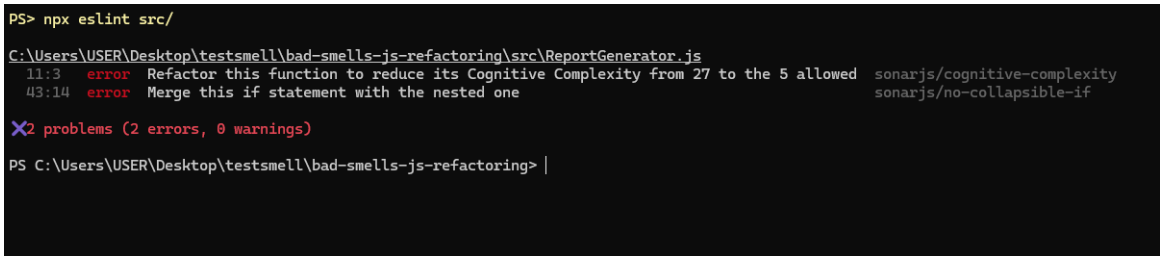


Figura 2 — npx eslint src/ no código original: 2 erros.

O plugin ajudou em dois pontos que a leitura manual não cobre bem:

- **Quantificação da complexidade.** A leitura manual indica que o método "parece complicado", mas a regra `sonarjs/cognitive-complexity` mede: **27**, mais de cinco vezes o limite. A métrica penaliza o aninhamento (`if` dentro de `if` dentro de `for`), o que mostra que o problema não é o tamanho do método, e sim a profundidade das decisões.
- **Detalhe estrutural.** A regra `sonarjs/no-collapsible-if` apontou na linha 43 um `if (item.value <= 500)` aninhado sem necessidade, um detalhe fácil de deixar passar na leitura.

A ferramenta não substitui a análise manual. Ela não apontou a duplicação das linhas, os números mágicos nem o efeito colateral, e a regra `no-identical-functions` só detecta funções inteiras duplicadas, não trechos. As duas abordagens se complementam: a ferramenta mede, e a leitura interpreta.

Observação sobre o ambiente: o projeto usa módulos ES (`"type": "module"`) e o `npm test` original não executava nenhum teste. Ajustei apenas o script no `package.json` para `node --experimental-vm-modules node_modules/jest/bin/jest.js`, sem alterar os testes.

3. Processo de Refatoração

O smell mais crítico era o **Método Longo com condicionais aninhadas e repetidas** (complexidade cognitiva 27), porque concentra os outros dois: é a mistura de tipo e perfil no mesmo laço que causa a duplicação.

Antes (corpo do laço em `src/ReportGenerator.js`)

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) {
      item.priority = true;
    }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? 'style="font-weight:bold;"' : '';
      report += `<tr ${style}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') {
        report += `${item.id},${item.name},${item.value},${user.name}\n`;
        total += item.value;
      } else if (reportType === 'HTML') {
        report += `<tr><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
        total += item.value;
      }
    }
  }
}
// + cabeçalho e rodapé com mais 4 comparações de reportType
```

Depois (`src/ReportGenerator.refactored.js`, trechos principais)

```
const ADMIN_PRIORITY_THRESHOLD = 1000;
const USER_MAX_VISIBLE_VALUE = 500;

const ROLE_POLICIES = {
  ADMIN: { canSee: () => true, isPriority: (item) => item.value > ADMIN_PRIORITY_THRESHOLD },
  USER:  { canSee: (item) => item.value <= USER_MAX_VISIBLE_VALUE, isPriority: () => false },
};

class CsvFormatter  { header() {...}  row(item, user) {...}              footer(total) {...} }
class HtmlFormatter { header(user) {...}  row(item, user, isPriority) {...}  footer(total) {...} }

const FORMATTERS = { CSV: new CsvFormatter(), HTML: new HtmlFormatter() };

generateReport(reportType, user, items) {
  const formatter = FORMATTERS[reportType];
  if (!formatter) {
    return '';
  }

  const policy = this.getPolicyFor(user);
  const visibleItems = items.filter((item) => policy.canSee(item));
  const rows = visibleItems.map((item) => formatter.row(item, user, policy.isPriority(item)));
  const total = this.calculateTotal(visibleItems);

  return [...formatter.header(user), ...rows, ...formatter.footer(total)].join('\n').trim();
}
```

Técnicas aplicadas

- **Replace Conditional with Polymorphism:** usei duas classes, `CsvFormatter` e `HtmlFormatter`, com a mesma interface (`header`, `row`, `footer`), selecionadas pelo mapa `FORMATTERS`. As 8 comparações de `reportType` foram eliminadas, e um formato novo agora é apenas uma nova classe.
- **Decompose Conditional / Strategy por perfil:** a regra de cada perfil virou um objeto em `ROLE_POLICIES` (`canSee`, `isPriority`). Isso também resolve o `no-collapsible-if`.
- **Extract Method:** usei `getPolicyFor` e `calculateTotal` para nomear cada etapa. O método principal passou de 58 para 13 linhas e agora se lê como uma descrição do processo.
- **Replace Loop with Pipeline + eliminação de duplicação:** com `filter → map → reduce`, a linha CSV e o cálculo do total passaram a existir em um único lugar.
- **Replace Magic Number with Symbolic Constant** para 1000 e 500. Também removi o efeito colateral: a prioridade é calculada, e o item de entrada não é mais alterado.

Métrica	Original	Refatorado
Complexidade cognitiva (maior método)	27	1
Erros do ESLint	2	0
Linhas de generateReport	58	13
Comparações de reportType	8	0
Ocorrências de total += item.value	4	1 (calculateTotal)
Testes passando	4/5 (5/5 após correção)	5/5

Tabela 1 — Comparação antes × depois.

4. Validação com a Rede de Segurança

Criei `tests/ReportGenerator.refactored.test.js` como cópia exata do teste original, mudando apenas o `import` (Figura 3). Rodei os testes após cada passo da refatoração. No arquivo original, corrigi somente a linha do `<tr >` apontada pela suíte. No final, as duas suítes passam (Figura 4) e o ESLint não acusa nenhum problema no arquivo refatorado (Figura 5).

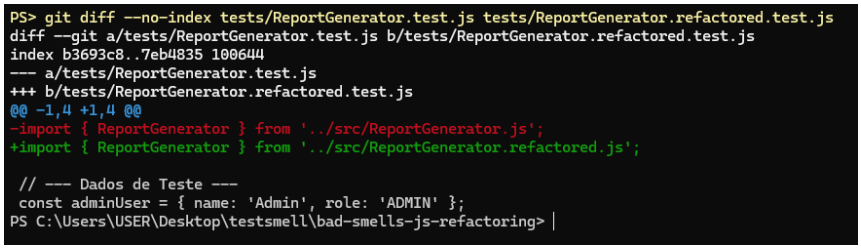


Figura 3 — Diferença entre os arquivos de teste: apenas o import.

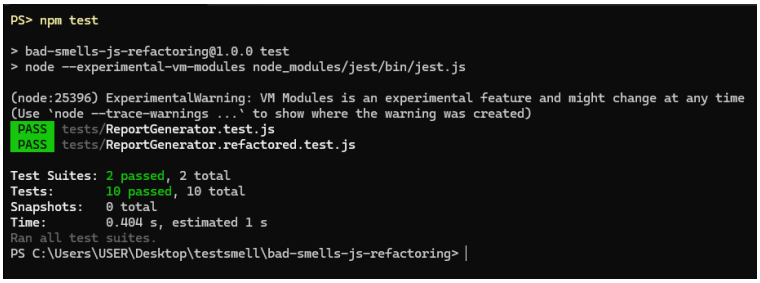


Figura 4 — npm test final: 2 suítes e 10 testes passando.

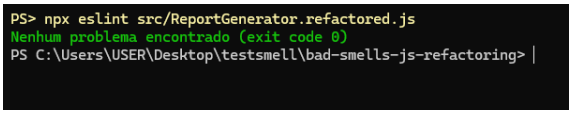


Figura 5 — npx eslint src/ReportGenerator.refactored.js sem problemas.

5. Conclusão

Refatorar é mudar a estrutura sem mudar o comportamento, e a única forma objetiva de provar isso é com testes que verificam a saída, e não a implementação. Foi essa suíte que permitiu trocar o método inteiro por classes, mapas e um pipeline com segurança: a cada passo, a confirmação de que nada quebrou vinha em menos de um segundo. Os testes também mostraram que a rede de segurança funciona antes mesmo da refatoração, ao revelar um defeito causado justamente pelo código duplicado.

A redução dos smells trouxe ganhos concretos. A complexidade cognitiva caiu de 27 para no máximo 1 por método. Cada regra de negócio tem um nome e um único lugar. E o código ficou aberto a extensão: um novo formato ou perfil se adiciona sem editar o que já funciona. O resultado é um software mais fácil de ler, de testar em partes e de evoluir. Usar a análise estática junto com a revisão manual torna esse processo mensurável e repetível.